

8.7 何为引脚功能复用？有何意义？

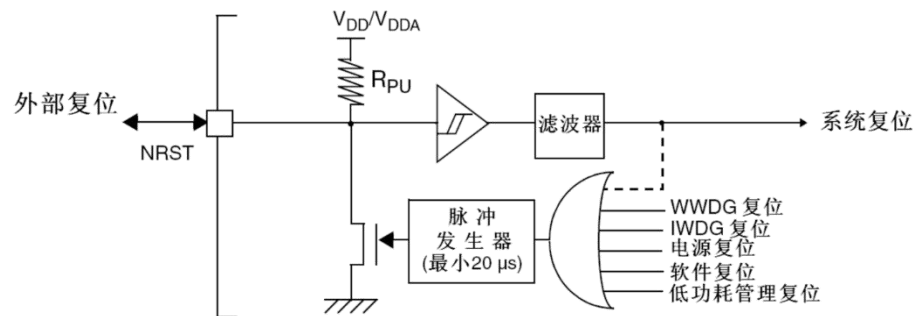
[解析] IO 口可以分配给不同的外设。“节约芯片引脚，可分时复用外设，虚拟增加端口数量，优化引脚配置和布线设计 PCB，同时减少信号交叉干扰”

8.11 请设计一个 STM32 最小系统。

[解析] 包含模块：电源、时钟、复位、调试。

8.12 STM32F103 的复位电路有何功能？常见的复位方式有哪些？

[解析] 复位电路使芯片进入预定义的初始化状态（即主要寄存器被设置为缺省值）。常见复位方式如下图所示，



8.14 STM32F103 的时钟有哪几种产生方式？各有什么特点？ =

[解析]

STM32 的时钟来源：内部（HSI、LSI），外部（HSE、LSE）

高速时钟源 HSE 和 HSI 可以经 PLL 倍频，提高时钟频率有助于提升性能；

低速时钟源 LSI 和 LSE 有利于降低功耗；

使用内部时钟源无需外接谐振单元电路，但时钟稳定度差。

8.13 STM32 的启动模式有几种？BOOT 引脚如何设置？

[解析]

[BOOT0, BOOT1]	启动方式	→0x0→MSP
0X	从用户 Flash 启动	0x08000000
10	从系统 Flash 启动	0x1FFFF000
11	从片内 SRAM 启动	0x20000000

8.16 参照 STM32 的时钟树，请做以下思考，并解释

- 1) 设计成这种形式的主要目的和理由为何？
- 2) 从左至右，相关时钟依次可分为大致哪几种？
- 3) 时钟输出的使能有何意义？

[解析]

- 1) 可以根据不同的外设，配置不同的时钟频率，且相互独立互不干扰，最大化系统性能；
- 2) 时钟源（外部/内部，高速/低速），锁相环，分频、倍频器，系统、外设时钟；
- 3) 功耗管理：在低性能需求场景，降低时钟频率，从而降低功耗。

8.21 以 STM32 为例，描述程序开发的一些主要模式，各自有什么特点，优势和缺点都体现在哪些地方？ =

[解析] 基于寄存器、基于库、基于 OS、基于代码生成工具。

优缺点可以从可移植性、可优化性、难易程度、硬件覆盖度等方面比较。

8.23 以 STM32 固件库为基础利用工程方式，进行嵌入式软件应用开发，有哪些主要步骤？请略述之。

[解析] 安装 IDE、安装 DFP、安装标准外设库、代码编写、编译链接、利用 Debugger 下载

[补充 8A-4] *PC 寄存器和 SP 寄存器都保存的是“地址”，二者有何本质区别？

[解析] PC 存放代码地址，SP 存放 Stack 指针（指向“数据”的地址）。

[补充 8A-5] *某 STM32F1 系列芯片启动后，地址“0x0000 0004”的存储单元内值为“0x0800 0800”，请问“0x0800 0800”代表了什么？

[解析] 系统启动时，从地址 0x0000 0004 处，取出复位异常服务程序入口地址放入 PC。

（课件第五章 5.5.1.1 P.194）0x0000 0004 是异常向量表的第二项（编号 1），是复位异常处理程序的入口地址。

[补充 8A-6] *某 STM32F1 系列芯片启动后，地址“0x0000 003C”的存储单元内值为“0x0800 0800”，请问“0x0800 0800”代表了什么？

[解析] $0x0000\ 003C = 60 = 15 \times 4$ ，是异常向量表的第 16 项（编号 15）SysTick 异常处理程序的入口地址。

[补充 8A-7] *查阅 F103 系列芯片参考手册（BB 系统→课程资源→参考资料→STM32 资料→STM32F1 reference manual），说明如下 C 代码的功能。

(volatile unsigned long)(0x40021004) |= 0x28000000 → 0x0500 0000;

[解析]

2.3 MemoryMap

起始地址	外设	总线	寄存器映像
0x4002 1000 - 0x4002 13FF	复位和时钟控制(RCC)	AHB	参见6.3.11节

6.3.11 0x40021_004

偏移	寄存器	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	RCC_CR	保留						PLLRDY	PLLON	保留				CSSON	HSEBYP	HSERDY	HSEON	HSICAL[7:0]							HSITRIM[4:0]				保留	HSIRDY	HSION		
	复位值							0	0					0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1
004h	RCC_CFGR	保留						MCO[2:0]		保留	USBPRE	PLLMUL[3:0]			PLLTPRE		PLLSRC	ADC PRE [1:0]	PRRE2 [2:0]		PRRE1 [2:0]		HPRE[3:0]			SWS[1:0]		SW[1:0]					
	复位值							0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

→ 6.3.2 Clock configuration register (RCC_CFGR)

0x05000000 = 0b 0000 0101 0000 ...

Bits 26:24 → MCO: Microcontroller clock output

Set and cleared by software.

0xx: No clock

100: System clock (SYSCLK) selected

101: HSI clock selected (HSI 作为时钟输出)

110: HSE clock selected

111: PLL clock divided by 2 selected

[补充 8A-8] *简述 STM32F103 的启动过程。

[解析]

根据 BOOT0 和 BOOT1 引脚，选择启动存储器映射

从地址 0x00000000 处，取出栈顶指针值放入 MSP

从地址 0x00000004 处，取出复位异常服务程序入口地址放入 PC

执行复位异常服务程序 Reset_Handler()

[补充 8A-9] 已知 STM32F1 系列单片机有如下 GPIO 相关的寄存器，按照 p190 第 4 章 4.4.1

小节“4. I/O 接口结构”部分的分类方式，这些寄存器各自属于什么类型的端口寄存器？

寄存器名称	类型
Port configuration register low (GPIOx_CRL)	控制寄存器
Port configuration register high (GPIOx_CRH)	控制寄存器
Port input data register (GPIOx_IDR)	数据寄存器
Port output data register (GPIOx_ODR)	数据寄存器
Port bit set/reset register (GPIOx_BSRR)	数据寄存器
Port bit reset register (GPIOx_BRR)	数据寄存器
Port configuration lock register (GPIOx_LCKR)	控制寄存器
Alternate function I/O and debug configuration (AFIO)	控制寄存器

[解析] 查阅手册对寄存器的定义，即可分类。

接口电路中端口寄存器也分为三类：①数据端口（寄存器）：用来存放 CPU 和外设间交换的数据，具有输入和输出数据缓冲功能。②状态端口（寄存器）：用来存放外设状态，如准备就绪（Ready）、忙碌（Busy）和错误（Error）等。③控制端口（寄存器）：用来存放 CPU 发往外设的控制命令，如启动、停止和复位等

[补充 8A-10] 根据“stm32f10x.h”中的定义（如下），

- （1）说明代码行①操作了 4GB 存储空间的哪些存储单元（给出地址范围）的功能。
- （2）说明代码行①完成的功能。

```
// .....\stm32f10x.h
```

```
typedef struct{
    vu32 CRL;
    vu32 CRH;
    vu32 IDR;
    vu32 ODR;
    vu32 BSRR;
    vu32 BRR;
    vu32 LCKR;
}GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOA_BASE (APB2PERIPH_BASE + 0x0800)
#define APB2PERIPH_BASE (PERIPH_BASE + 0x10000)
#define PERIPH_BASE ((uint32_t)0x40000000)
GPIOA->BSRR = 0x10000030; //①
```

[解析]

```
(1) 0x4001 0810 ~ 0x4001 0813
= 0x40000000 + 0x10000 + 0x0800 + (4*4bytes)
= GPIOA + (4 * sizeof(vu32))
```

结构体中元素的定义顺序与存储顺序一致。BSRR 为第五项，在结构体内地址偏移即为 $(4 * \text{sizeof}(vu32))$ 。

(2) 查询手册。BSRR 高 16bits 用于 Reset，低 16 位用于 Set。0x10000030 对应第 29、6、5 位置 1，故①完成的功能是：复位（清零/Reset）PA12；同时置位（置 1/Set）PA4 和 PA5。

[补充 8A-11] 根据“stm32f10x.h”中的定义（如上题），（1）说明代码行①操作的 4GB 存储空间的哪些存储单元（给出地址范围）的功能。（2）说明代码行①和②完成的功能。

```
GPIOA->ODR |= 1<<7;           //①
```

```
GPIOA->ODR &= ~(1<<6));       //②
```

[解析] ODR Output Data Register

(1) $0x4001\ 080C \sim 0x4001\ 080F = 0x40000000 + 0x10000 + 0x0800 + (3*4\text{bytes})$

①和②PA7 置位（置 1/输出高电平）

(2) ①完成 PA7 置位（置 1/输出高电平）

②完成 PA6 清零（置 0/输出低平），等同于 $\text{GPIOA_ODR} \&= 0xBF(1011\ 1111B)$

[补充 8A-12] *根据“stm32f10x.h”中的定义（如上题），说明如下代码的功能。

```
GPIOA->ODR = *(volatile unsigned long*)(0x00000000);
```

[解析] 将主栈指针寄存器 MSP 内的值写入 GPIOA 的输出数据寄存器 ODR，即根据 MSP 内的数据控制 PA0~PA15 的引脚输出电平。

8.31 简述通用定时器的输入捕获过程。

[解析]检测 TIMx_CHx 引脚信号，在信号发生跳变（比如上升沿/下降沿）时，将定时器计数值（TIMx_CNT）存放到对应的捕获/比较寄存器（TIMx_CCRx），完成一次捕获。

同时，还可以配置是否触发中断/DMA 等。

8.34 简述通用定时器的比较输出过程。

[解析]

当定时器计数值（TIMx_CNT）变动至与捕获/比较寄存器（TIMx_CCRx）值相等时，在 OCx 输出引脚输出特定信号，输出信号依据设置可能是：置位、复位、翻转或不变。

同时，还可以配置是否触发中断/DMA 等

8.36 利用 TIM2 中断，设计一个红绿灯系统，要求：红灯亮 5 秒钟，熄灭，切换绿灯亮 5 秒钟，熄灭，循环往复。

给出硬件原理图，并编写代码实现。

[解析]

答案不唯一，合理即可。

```
void TIM2_IRQHandler() //中断服务子程序
```

```
{
    if(TIM_GetITStatus(TIM2,TIM_IT_Update!=RESET)
    {
        if(flag)
        {
            LED_GREEN_OFF(); //对 PB8 写 0
            LED_RED_ON();    //对 PA18 写 1
        }
        else
        {
```

```

        LED_RED_OFF(); //对 PA18 写 0
        LED_GREEN_ON(); //对 PB8 写 1
    }
    flag = !flag; //反转 flag，这样下一次触发中断时亮不同的灯
}
}
int main()
{
    int flag =0; //这是中断服务程序中指示当前亮的灯种类的变量
    LED_RED_Config(); //对 PA18 进行配置
    LED_GREEN_Config(); //对 PB8 进行配置
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置中断优先级
    TIM2_Init(83999,4999); //TIM2 初始化，配置 arr 和 psc 使计数周期为 5s
    并使能中断
    while(1) {}
}

```

8.41 STM32F103 的中断系统共支持多少个异常？

其中包括多少个内部异常和多少个可屏蔽的非内核异常中断？

[解析]

支持 84 个异常，包括 16 个内部异常和 68 个可屏蔽的非内核异常中断。

8.43 STM32F103 的外部中断/事件线是如何与端口映射的？

[解析]

除 3 个特定中断外，EXTI0~15 连接/映射方式如下：

任一端口 0 号引脚(如 PA0、PB0、…、PG0)映射到 EXTI0 上，

任一端口 1 号引脚(如 PA1、…、PG1)映射到 EXTI1 上…，

任一端口 15 号引脚(如 PA15、…、PG15)映射到 EXTI15 上。

同一时刻，只能有一个端口(即 A~G 中仅选一个)的 n 号引脚映射到 EXTI_n 上(n 为 0~15)。

[补充 8A-13] 假设 TIMxCLK 为 72MHz，有如下代码。请问：

(1) 启动 TIM2 多长时间后 TIM2_CNT.CNT 数值为 720？

(2) 启动 TIM2 多长时间后 TIM2_ARR.ARR 与 TIM2_CNT.CNT 数值相同？

```

#include stm32f10x_tim.h

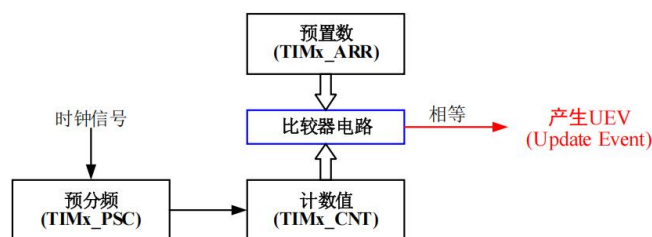
void main()
{
    //.....

    TIM_TimeBaseInitTypeDef my_timer;
    my_timer.TIM_Prescaler= 71999 ; //实际取值 < 216=65536
    my_timer.TIM_Period= 999 ;
    my_timer.TIM_CounterMode= TIM_CounterMode_Up ;
    TIM_TimeBaseInit (TIM2, & my_timer) ;
    //.....
}

```


[解析]

(1)



CNT 计数频率 = $TIMxCLK / (PSC + 1) = 72M / 72k = 1000Hz$

$T = 720 / 1000 = 0.72s$.

(2) 定时周期(秒) = 预置数 * 分频系数 / 时钟频率

$Tout(s) = ((ARR + 1) * (PSC + 1)) / TIMxCLK = (1,000 * 72,000) / 72,000,000 = 1s$.

[补充 8A-14] * 阅读教材 P558 (或 Cortex™-M3 Devices Generic User Guide) 关于 SysTick 各寄存器位定义的描述，分析如下代码。假设 HCLK 为 24MHz，请问：

(1) 代码行①、②、③的作用。

(2) 代码行④中的 ui_ticks 取值应为多少？

```
#define SysTick_CSR(*((volatile unsigned int*) 0xE000E010)) //控制寄存器
#define SysTick_LOAD(*((volatile unsigned int*) 0xE000E014)) //重载寄存器
#define SysTick_VAL(*((volatile unsigned int*) 0xE000E018)) //当前值寄存器
#define SysTick_CALRB(*((volatile unsigned int*) 0xE000E01C)) //校准值寄存器
SysTick_Init(unsigned int ui_ticks)
{
    SysTick_VAL = 0;           //当前值寄存器清零
    SysTick_CSR |= 0x4;        //①
    SysTick_CSR |= 0x2;        //②
    SysTick_CSR |= 0x1;        //③
    SysTick_LOAD = ui_ticks;   //④
}
```

[解析]

代码行①，CLKSOURCE=1，选择时钟源为 HCLK/8=4MHz (HCLK 参见教材 P469 图 8.21)

代码行②，TICKINT=1，使能中断

代码行③，ENABLE=1，使能 SysTick

代码行④， $(ui_ticks + 1) \times (1 / (3 \times 10^6)) = 10 \times 10^{-3} \rightarrow ui_ticks = 29999$

[补充 8A-15] * 已知 STM32F407 实验箱上 LED 灯 D0 连接 PG10，开关 DIP0 连接 PE4。设计程序，实现 DIP0 开关拨动的时候触发中断，在中断服务子函数中控制灯的亮灭。请给出 main() 函数代码和中断服务子程序代码。

[解析]

main() 函数应包括 PG10 输出的 GPIO 初始化、PE4 输入的 GPIO 初始化、EXTI 初始化和 NVIC 初始化。

中断服务子程序应为 EXTI4_IRQHandler()。

8.50 简述 STM32F103 的 USART 数据接收/发送基本过程。

[解析]

发送过程：CPU 写 TDR 寄存器，TDR 数据存入发送移位寄存器后 SR.TXE 置 1，但此时发送并未结束，发送完毕时，SR.TC 置 1。同时，可配置产生 TC、TXE 等事件触发 USART 中断。

接收过程：RXD 引脚接收到串行数据，一位一位的数据移位进入接收移位寄存器，SR.RXNE 置 1，CPU 读 RDR 寄存器。同时，可配置产生 RXNE、NE 等事件触发 USART 中断。

[补充 8A-16] 解释如下寄存器位的含义。

寄存器位	含义
USART_SR.TXE	TDR 数据已经存入发送移位寄存器
USART_SR.TC	发送移位寄存器中各个比特发送完毕
USART_SR.RXNE	接收到数据，缓冲区非空

[补充 8A-17] *在 USART 收发数据的过程中，会产生 TC、TxNE、RxNE 等不同事件，然而每个 USART 模块只对应一个中断向量。在中断服务程序中需要做什么处理？

[解析]

需要在中断服务子程序中读取相关标志位以判断产生的什么事件。

USART_SR[7] TXE: Transmit data register empty

USART_SR[6] TC: Transmission complete

USART_SR[5] RXNE: Read data register not empty

可能的处理代码形式如下：

//例如，判断发送是否完成(TC)，返回值 SET 或 RESET，即 1/0

```
if (USART_GetFlagStatus(USART1, USART_FLAG_TC) == SET) { //已发完,
    SR.TC == 1; }
```

//例如，判断是否接收到数据

```
if (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) != RESET)
{ //接收缓冲区非空, SR.RXNE == 1 }
```

[补充 8A-18] *解释如下结构体各成员变量的含义。

typedef struct

{

uint32_t USART_BaudRate; //波特率

uint16_t USART_WordLength; //数据帧的字长，即每次发送/接收的数据位数

uint16_t USART_StopBits; //停止位数

uint16_t USART_Parity; //校验方式

uint16_t USART_HardwareFlowControl; //硬件流控

uint16_t USART_Mode; //通信模式

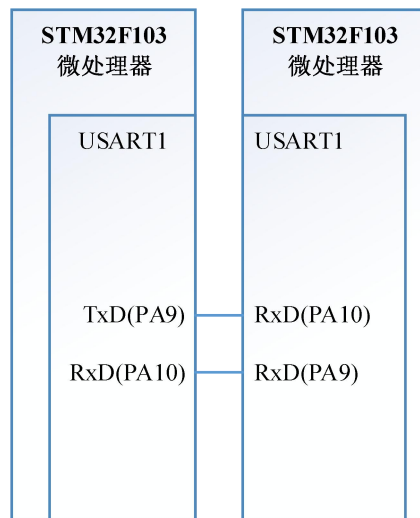
} USART_InitTypeDef;

```
54 typedef struct
55 {
56     uint32_t USART_BaudRate; /*< This member configures the USART communication baud rate.
57                               The baud rate is computed using the following formula:
58                               - IntegerDivider = ((PCLKx) / (8 * (OVR8+1) * (USART_InitStruct->USART_BaudRate)))
59                               - FractionalDivider = ((IntegerDivider - ((u32) IntegerDivider)) * 8 * (OVR8+1)) + 0.5
60                               Where OVR8 is the "oversampling by 8 mode" configuration bit in the CR1 register. */
61
62     uint16_t USART_WordLength; /*< Specifies the number of data bits transmitted or received in a frame.
63                               This parameter can be a value of @ref USART_Word_Length */
64
65     uint16_t USART_StopBits; /*< Specifies the number of stop bits transmitted.
66                               This parameter can be a value of @ref USART_Stop_Bits */
67
68     uint16_t USART_Parity; /*< Specifies the parity mode.
69                               This parameter can be a value of @ref USART_Parity
70                               @note When parity is enabled, the computed parity is inserted
71                               at the MSB position of the transmitted data (9th bit when
72                               the word length is set to 9 data bits; 8th bit when the
73                               word length is set to 8 data bits). */
74
75     uint16_t USART_Mode; /*< Specifies whether the Receive or Transmit mode is enabled or disabled.
76                               This parameter can be a value of @ref USART_Mode */
77
78     uint16_t USART_HardwareFlowControl; /*< Specifies whether the hardware flow control mode is enabled
79                               or disabled.
80                               This parameter can be a value of @ref USART_Hardware_Flow_Control */
81 } USART_InitTypeDef;
```

8.52 *设计一个简单的双机通信系统，要求：在两个 ARM 处理器之间，用 USART 实现，

各自具有收/发缓冲内存区域，协议自定，结束条件自定。给出相关硬件连接图，并编写核心代码。

[解析]



答案不唯一，合理即可。以下给出的是单工方式，即一端只发，另一端只接收。

//发送端

```
int main (void) {
    unsigned char uc_send_ch = 0, uc_rcv_ch = 0;
    GPIOA_Config(); //初始化 GPIOA
    USART1_Config (9600); //USART1 Init with 9600bps
    printf ("USART1 start...\n");
    while (1) {
        TIM2_Delay125MS();
        USART_SendData (USART1, uc_send_ch++); //发送字符
        printf("Send:0x%x\n",uc_send_ch);
    }
}
```

//接收端

```
int main (void) {
    unsigned char uc_send_ch = 0, uc_rcv_ch = 0;
    GPIOA_Config(); //初始化 GPIOA
    USART1_Config (9600); //USART1 Init with 9600bps
    while (1) {
        TIM2_Delay125MS();
        if (USART_GetFlagStatus (USART1, USART_FLAG_RXNE) !=RESET){
            uc_rcv_ch = USART_ReceiveData(USART1); //接收字符
            printf("Receive: 0x%x\n",uc_rcv_ch);
        }
    }
}
```

4.61 描述 I²C 总线协议中的状态，并画出状态转移图。

[解析]

总线空闲 I: SDA 与 SCL 均为高电平

启动数据传输 S: SCL 为高电平，SDA 负跳变

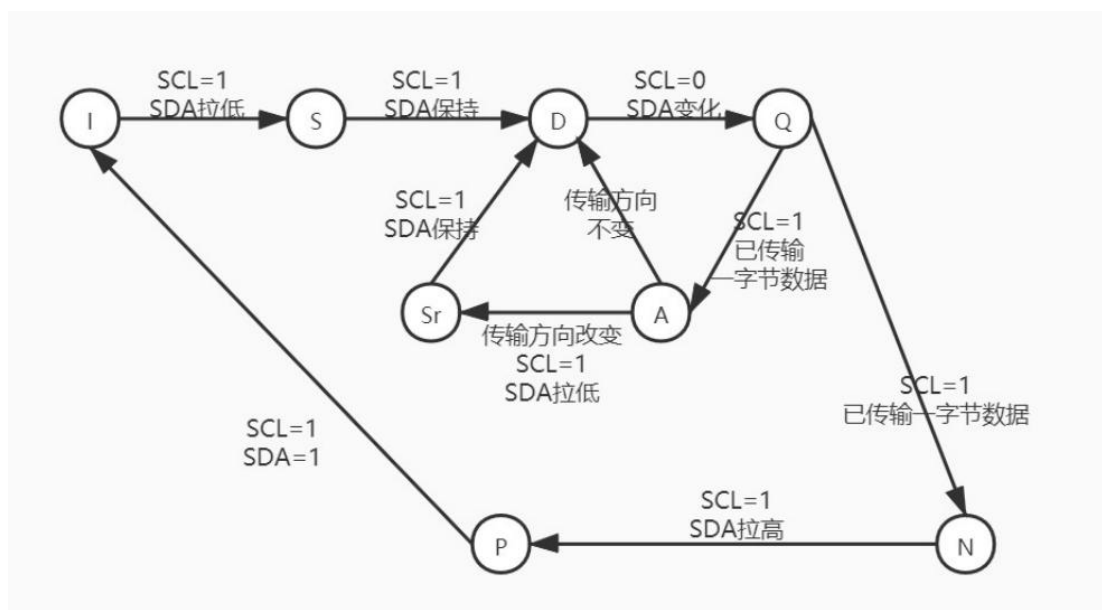
停止数据传输 P: SCL 为高电平，SDA 正跳变

重新启动 Sr: SCL 为高电平，SDA 负跳变

数据有效 D: SCL 为高电平期间，SDA 电平稳定

等待/数据无效 Q: SCL 保持低电平，此时可修改线上数据

应答 A/不应答 N: 接收方将 SDA 拉低发出 ACK 或释放 SDA 发出 NACK



[补充 4.64] 对比“I²C 总线规范中定义的地址”、“访问存储器的地址”、“Cache 地址”、“I/O 端口地址”有何不同？

[解析] 大致描述出以下信息即可，

“I²C 总线规范中定义的地址”是 I²C 器件地址，即器件的 ID，用于指示被访问的从器件。

“访问存储器的地址”是存储器内存储单元的 ID，用于指示拟访问的存储单元。

“Cache 地址”是 Cache 内存储单元的 ID。

“I/O 端口地址”是 I/O 接口中（控制、数据、状态）寄存器的 ID，用于指示被访问的寄存器。

[补充 4.66] 以下关于 SPI 总线说法正确的是（ ）

- I. SPI 是一种全双工通信总线，即主设备读从设备、主设备写从设备可同时进行
- II. 采用 SPI 通信时，主设备和从设备是共用时钟信号的
- III. SPI 主设备通过 CS# 信号指示从设备数据传输的开始与结束
- IV. SPI 是同步通信总线
- V. SPI 总线是串行通信总线
- VI. 从设备向主设备发送数据既可以在时钟上升沿，也可以在时钟的下降沿
- VII. 接收设备采样数据线的总是滞后发送设备推送数据半个周期

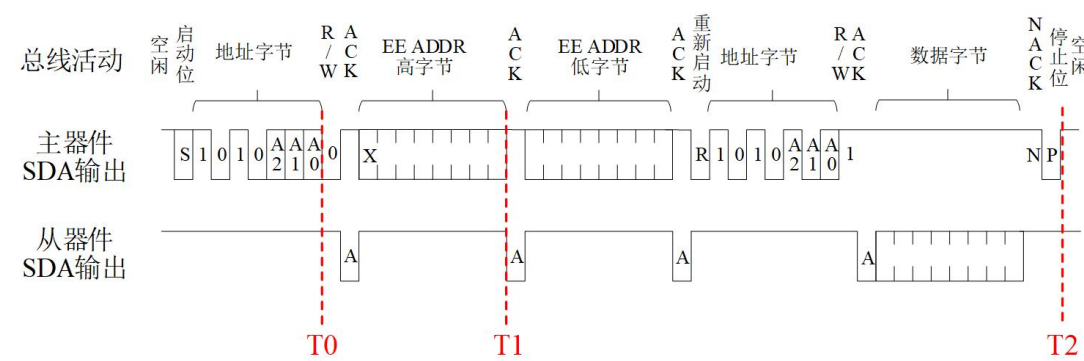
- A) 仅 I、II、III、IV
- B) 仅 II、III、IV、V
- C) 仅 III、IV、V、VI
- D) 仅 IV、V、VI、VII
- E) 以上答案均不正确

[解析] E。
所有说法都是正确的

[补充 8A-19] *解释如下寄存器位的含义。

寄存器位	含义
SPI_CR2.TXEIE	Tx buffer empty interrupt enable
SPI_CR2.RXNEIE	RX buffer not empty interrupt enable
SPI_SR.TXE	Transmit buffer empty
SPI_SR.RXNE	Receive buffer not empty

[补充 8A-20] *在图 4.84 的不同时刻（如下图所示 T0 时刻、T1 时刻、T2 时刻），I²C 主设备和 I²C 从设备各产生哪些事件？从 I2C_SR1.SB、I2C_SR1.ADDR、I2C_SR1.STOPF、I2C_SR1.BTF、I2C_SR1.RXNE、I2C_SR1.TXE 中选择。



时刻	主设备事件	主设备标志位	从设备事件	从设备标志位
T0	发送启动条件和地址字节	I2C_SR1.SB = 1	等待地址匹配	无
T1	地址发送完成 收到 ACK	I2C_SR1.ADDR = 1 I2C_SR1.TXE = 1	接收地址 发送 ACK	I2C_SR1.ADDR=1 I2C_SR1.RXNE = 1
T2	发送停止条件	I2C_SR1.STOPF = 1	检测到停止条件	无